

Cyber Physical Systems

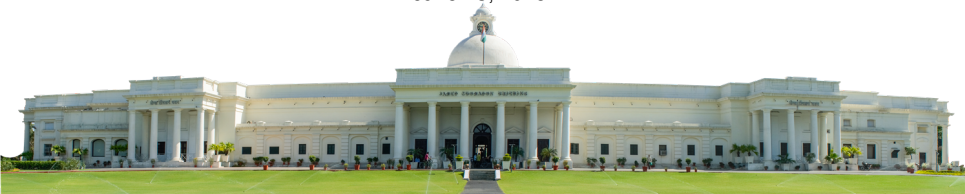
Basics of TinyML

Sambit Sahoo
Intern, CoDA Lab

Under the guidance of
Dr. Sudip Roy, Dept. of CSE, IIT Roorkee

sambit22@iiserb.ac.in
GitHub Link

June 13, 2025





1 Arduino

2 Parts of the Arduino UNO Board

3 Functions

4 Basic Experiments

- Blinking External LED
- Morse Code SOS Flasher
- Morse Code Translator
- Pushbutton and LED
- Using Potentiometer
- Servo Motor with Arduino
- Control Servo Motor with Potentiometer
- Ultrasonic Sensor HC-SR04
- Temperature Monitoring System
- Bluetooth Controlled Car



1 **Arduino Intermediate Experiments**

- DHT11 Sensor Values in LCD
- Home Automation Project (Fans, Light)
- Smart Street Lamps
- Motor Control with Auto and Manual Mode
- Traffic Light Control and Seven Segment Display
- Controlling lights with Joystick Module
- Controlling Servo Motor using Joystick
- Smart Irrigation System
- Joystick Controlled Robotic Arm
- 4x4 Keypad Module with LCD



1 **Arduino Hands-On Projects**

- Smart Door Lock System
- DHT Values in LCD
- Automatic Street Lamps using LDR
- LED Brightness Control with Potentiometer
- Increment and Decrement Counter using Pushbuttons with Arduino
- Traffic Light Control and Seven Segment Display
- Arduino Door Security System using IR Sensor



1 Machine Learning

- Supervised Learning
- Blockflow diagram of Supervised ML
- Unsupervised Learning

2 Deep Learning

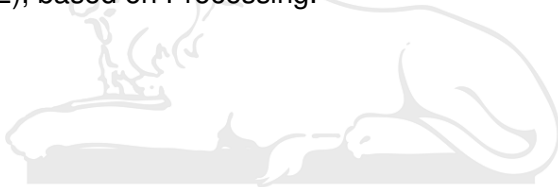
- Artificial Neural Network (ANN)
- Convolutional Neural Network (CNN)

3 TinyML

- Introduction
- TinyML Workflow



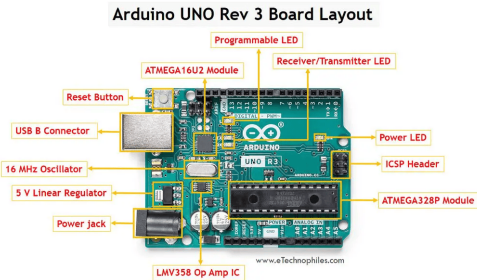
Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board. To do so you use the Arduino programming language (based on Wiring), and the Arduino Software (IDE), based on Processing.



Parts of the Arduino UNO Board



- ❑ Microcontroller
- ❑ Digital I/O Pins (0–13)
- ❑ Analog Input Pins (A0–A5)
- ❑ Power Pins
- ❑ USB Port
- ❑ Voltage Regulator
- ❑ Crystal Oscillator (16 MHz)
- ❑ Reset Button
- ❑ Power LED (ON)
- ❑ TX/RX LEDs
- ❑ ICSP Header (In-Circuit Serial Programming)
- ❑ Bootloader





Microcontroller

- ☐ The brain of the Arduino.
- ☐ Executes the program (sketch) uploaded from your computer.
- ☐ Contains memory (Flash, SRAM, EEPROM), clock, timers, and I/O pins.

Digital I/O Pins

- ☐ Can be used to read or send digital signals (HIGH/LOW).
- ☐ Pins 0 and 1 are also used for serial communication (RX/TX).

Analog Input Pins (A0–A5)

- ☐ Used to read analog sensors (e.g., temperature sensors, potentiometers).
- ☐ Convert analog voltage (0–5V) to digital values (0–1023) using an ADC.



Power Pins

- ☐ **Vin**: Input voltage if using an external power source.
- ☐ **5V**: Regulated 5V supply from USB or onboard regulator.
- ☐ 3.3V output (lower power sensors).
- ☐ **GND (Ground)**: Common ground reference.

USB Port

- ☐ Used to connect to a computer for Uploading code and Serial communication.
- ☐ Also powers the board when connected.

Voltage Regulator

- ☐ Regulates voltage down to a stable 5V or 3.3V to protect components.
- ☐ Works when external power is used (e.g., battery or adapter).



Crystal Oscillator (16 MHz)

- ☐ Provides the clock signal that keeps the microcontroller running at a consistent speed.

Reset Button

- ☐ Resets the microcontroller.
- ☐ Useful for restarting the program from the beginning without unplugging the board.

Power LED

- ☐ Lights up when the board is powered.

TX/RX LEDs

- ☐ Indicate data transmission (TX) and reception (RX) over USB or serial.

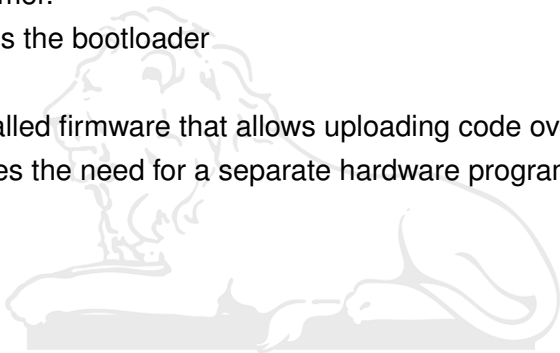


ICSP Header (In-Circuit Serial Programming)

- ☐ Used to program the microcontroller directly using an external programmer.
- ☐ Bypasses the bootloader

Bootloader

- ☐ Pre-installed firmware that allows uploading code over USB..
- ☐ Eliminates the need for a separate hardware programmer.



1. Blinking External LED



Blinking an external LED is often the first project when learning microcontrollers like Arduino. It demonstrates basic digital output control and timing.

- ❑ An LED is connected to one of the Arduino's digital pins via a current-limiting resistor.
- ❑ The Arduino turns the LED ON and OFF repeatedly using `digitalWrite()`.
- ❑ The `delay()` function controls the time intervals between ON and OFF states.
- ❑ This project introduces fundamental concepts:
 - ❑ Configuring pin modes with `pinMode()`.
 - ❑ Writing digital signals.
 - ❑ Using delays to create visible blinking.
- ❑ It is analogous to "Hello World" in programming — a simple, clear introduction to hardware interaction.

1. Blinking external LED



blinking_ext_LED | Arduino IDE 2.3.4

File Edit Sketch Tools Help



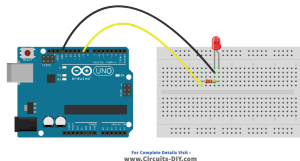
Arduino Uno



blinking_ext_LED.ino

```
1  //(Basic LED Blink Example)
2  const int ledPin = 13; // Pin where LED is connected
3
4  void setup() {
5    pinMode(ledPin, OUTPUT); // Set the LED pin as output
6  }
7
8  void loop() {
9    digitalWrite(ledPin, HIGH); // Turn the LED on
10   delay(1000);                // Wait for 1 second
11   digitalWrite(ledPin, LOW);  // Turn the LED off
12   delay(1000);                // Wait for 1 second
13 }
14
15
```

Arduino LED Blink Circuit



2. Morse Code SOS Flasher



Morse code is a method of encoding textual information using sequences of short and long signals, known as "dots" and "dashes." The SOS signal in Morse code is denoted by:

... — ...

This signal is universally recognized as a distress call. In the Morse Code SOS Flasher experiment using an Arduino:

- ❑ An LED is used to flash the SOS pattern.
- ❑ The timing of dots (short flashes) and dashes (long flashes) is precisely controlled using code.
- ❑ This experiment helps in understanding digital output control and timing functions.

2. Morse Code SOS Flasher



```
morsecode_SOS_flasher | Arduino IDE 2.3.4
File Edit Sketch Tools Help

[Check] [Next] [Previous] [Search] Arduino Uno

morsecode_SOS_flasher.ino
1  int ledPin = 13
2  int durations[] = {200,200,200,500,500,500,200,200,200}
3
4  void setup() {
5      pinMode(ledPin, OUTPUT); // put your setup code here, to run once:
6  }
7
8  void flash(int duration) {
9      digitalWrite(ledPin, HIGH);
10     delay(duration);
11     digitalWrite(ledPin, LOW);
12     delay(duration);
13 }
14
15 void loop() {                                // put your main code here, to run repeatedly:
16     for (int i = 0; i<9; i++){
17         flash(durations[i]);
18     }
19
20     delay(1000);                               // wait 1 second before we start
21 }
--
```

3. Morse Code Translator



A Morse Code Translator converts regular text into Morse code, which uses a combination of dots (.) and dashes (–) to represent letters, digits, and punctuation.

- ☐ Each character is mapped to its corresponding Morse code equivalent.
- ☐ The Arduino reads input characters (typically from a serial monitor or sensor).
- ☐ It then translates each character and outputs Morse code using:
 - ☐ LED blinks (short/long flashes),
 - ☐ Sound signals (buzzer beeps), or
 - ☐ Serial printouts.
- ☐ Timing is essential:
 - ☐ Dot: 1 time unit
 - ☐ Dash: 3 time units
 - ☐ Space between parts of the same letter: 1 unit
 - ☐ Space between letters: 3 units
 - ☐ Space between words: 7 units

3. Morse Code Translator



morsecode_translator | Arduino IDE 2.3.4

File Edit Sketch Tools Help

Arduino Uno

morsecode_translator.ino

```
1  int ledPin = 13;
2
3  char* letters[] = {
4    ".-", "...", "-.-.", "...", "...", "-.-.", "-.-.", "...", "...", // A-I
5    "-.-.", "-.", "...", "...", "...", "-.-.", "-.-.", "...", "...", // J-R
6    "...", "...", "...", "...", "...", "...", "...", "...", // S-Z
7  };
8
9  char* numbers[] = {"-----", "-.----", ".-.-.-", "...--", "-....-", ".....", "-.....",
10    "-.-..", "-.-.-", "-.-.-."};
11
12  int dotDelay = 200;
13
14  void setup(){
15    pinMode(ledPin, OUTPUT);
16    Serial.begin(9600);
17  }
18
19  void loop() { // put your main code here, to run repeatedly:
20    char ch;
21    if (Serial.available()) { // is there anything to read from USB?
22      char ch = Serial.read(); // read a single character
23
24      if (ch >= 'a' && ch <= 'z') {
25        flashSequence(letters[ch - 'a']);
26      }
27      else if (ch >= 'A' && ch <= 'Z') {
28        flashSequence(letters[ch - 'A']);
29      }
30      else if (ch >= '0' && ch <= '9') {
31        flashSequence(numbers[ch - '0']);
32      }
33    }
```

3. Morse Code Translator (contd..)

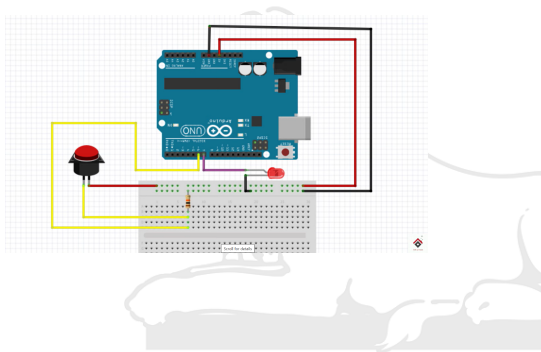


```
morsecode_translator | Arduino IDE 2.3.4
File Edit Sketch Tools Help

[Check] [Run] [Serial] Arduino Uno

morsecode_translator.ino
29     }
30     else if (ch >= '0' && ch <= '9') {
31         flashSequence(numbers[ch - '0']);
32     }
33     else if (ch == ' ') {
34         delay(dotDelay * 4);           // space between words
35     }
36 }
37 }
38
39 void flashSequence(char* sequence) {
40     int i = 0;
41     while (sequence[i] != '\0') {      // loop through each symbol in Morse code
42         flashDotOrDash(sequence[i]);
43         i++;
44     }
45     delay(dotDelay * 3);              // gap between letters
46 }
47
48 void flashDotOrDash(char dotOrDash) {
49     digitalWrite(ledPin, HIGH);
50     if (dotOrDash == '.') {
51         delay(dotDelay);              // dot = 1 unit
52     } else {
53         delay(dotDelay * 3);          // dash = 3 units
54     }
55     digitalWrite(ledPin, LOW);
56     delay(dotDelay);                  // gap between dots and dashes
57 }
58
59 }
60 }
```

4. Pushbutton and LED (using digitalRead)

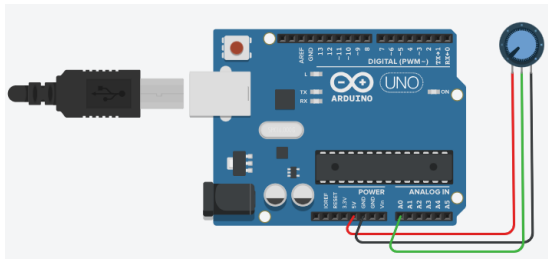


pushbutton_digitalread.ino

```
1  int s = 6;
2  int LED = 7;
3  int x;
4
5  void setup() {
6      serial.begin(9600);
7      pinMode(6, INPUT);
8      pinMode(7, OUTPUT);
9  }
10
11 void loop() {
12     x = digitalRead(s);
13     serial.println(x);
14     delay(1000);
15
16     if (x == 0){
17         digitalWrite(LED, LOW);
18     }
19
20     else{
21         digitalWrite(LED, HIGH);
22     }
23 }
```

- Serial is used because Arduino communicates data to the computer via serial communication. To start serial communication we have to set the baud rate (no. of bits transferred /second)

5. Potentiometer (using analogRead)

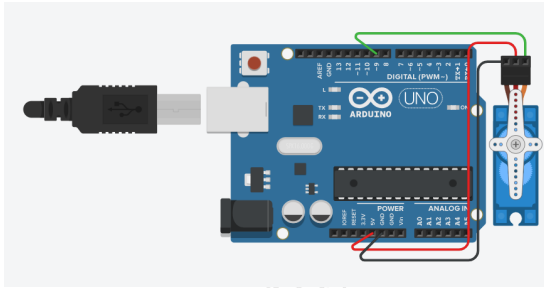


pot.ino

```
1 int pot = A0;
2 int x;
3
4 void setup(){
5     Serial.begin(9600);
6     pinMode(A0, INPUT);
7 }
8
9 void loop(){
10     x = analogRead(pot);
11     Serial.println(x);
12     delay(300);
13 }
```

- ❑ Demonstrates how to read analog input using 'analogRead()' from a variable resistor (potentiometer).
- ❑ Output value ranges from 0 to 1023, representing voltage between 0V to 5V.

6. Servo Motor with Arduino

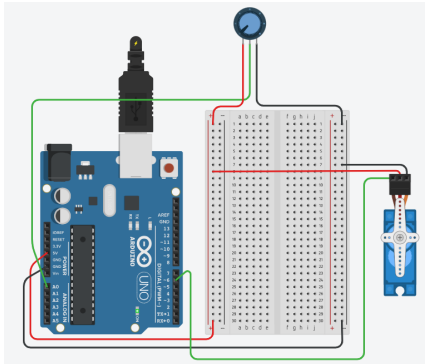


servo_motor.ino

```
1
2  #include <Servo.h>
3  Servo s1;
4
5  void setup(){
6      s1.attach(9);
7  }
8
9  void loop(){
10     for (int i = 0; i<= 180; i++){
11         s1.write(i);
12         delay(15);
13     }
14     for (int i = 180; i>=0; i--){
15         s1.write(i);
16         delay(15);
17     }
18 }
```

- ❑ Servo motors are used for precise control of angular position.
- ❑ Controlled using PWM signals; the angle is set via 'servo.write()' in Arduino.

7. Control Servo Motor with Potentiometer

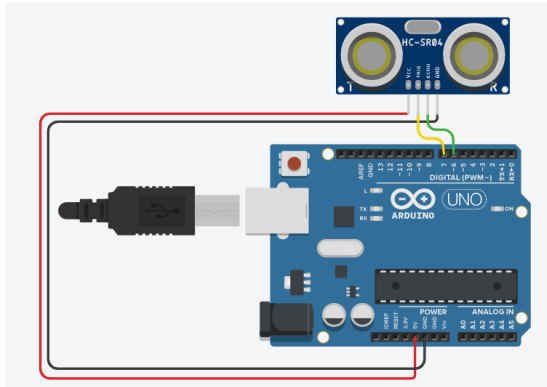


pot_servomotor.ino

```
1  #include <Servo.h>
2  int pot = A0;
3  Servo s1;
4  int x;
5  int value;
6
7  void setup(){
8      Serial.begin(9600);
9      s1.attach(6);
10     pinMode(A0, INPUT);
11 }
12
13 void loop(){
14     x = analogRead(pot);
15     value = map(x,0,1023,0,180);
16     s1.write(value);
17 }
18 }
```

- ❑ The 'map()' function converts the potentiometer's analog input range (0–1023) to a suitable angle range (e.g., 0–180°) for servo rotation.
- ❑ Similar experiment – Potentiometer controlling LED brightness (uses PWM concept).

8. Ultrasonic Sensor HC-SR04 with Arduino

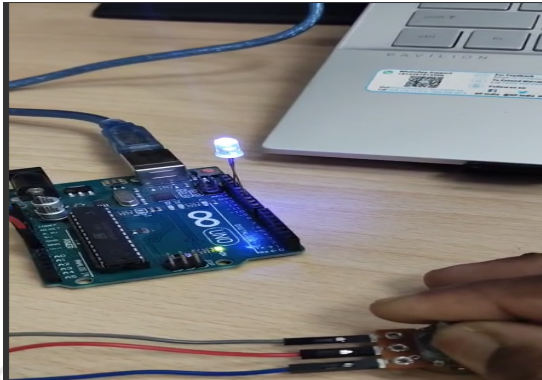


ultrasonic_sensor.ino

```
1 int trig = 7;
2 int echo = 6;
3 int timeInmicro;
4 int distanceIncm;
5
6 void setup(){
7   Serial.begin(9600);
8   pinMode(trig, OUTPUT);
9   pinMode(echo, INPUT);
10 }
11
12 void loop(){
13   digitalWrite(trig, LOW);
14   delay(200);
15   digitalWrite(trig, HIGH);
16   delay(200);
17   digitalWrite(trig, LOW); //pulse generated
18
19   timeInmicro = pulseIn(echo, HIGH);
20   distanceIncm = timeInmicro/29/2;
21   Serial.println(timeInmicro);
22 }
```

- ❑ HC-SR04 measures distance using ultrasonic sound waves and echo timing.
- ❑ 'pulseIn()' returns the time (μ s) the echo pin stays HIGH.
- ❑ Distance is calculated using: $\text{distance} = \text{time} / 29 / 2$.

4. LED Brightness Control with Potentiometer



□ [Link for Video Demonstration in GitHub](#)

9. Temp Monitor System using DHT and Motor



temp_monitor_system_dht_motor.ino

```
1  #include "DHT.h"
2  #define DHTPIN 2
3  #define DHTTYPE DHT11
4  DHT dht (DHTPIN, DHTTYPE);
5
6  int in1 = 2;
7  int in2 = 3;
8  int en1 = 6;
9  int y;
10
11 void setup() {
12   Serial.begin(9600);
13   dht.begin();
14
15   pinMode(in1,OUTPUT);
16   pinMode(in2,OUTPUT);
17   pinMode(en1,OUTPUT);
18 }
19
20 void loop() {
21   delay(2000);
22   float t = dht.readTemperature();
23   //checking if sensor is reading inputs
24   if (isnan(t)){
25     Serial.println("Failed to read from DHT");
26     return;
27   }
28
29   digitalWrite(in1,HIGH);
30   digitalWrite(in2,LOW);
31   y = map(t,20,50,0,255);
32   analogWrite(en1,y);
33 }
```

- ❑ **DHT11 Sensor:** Measures temperature and sends data to Arduino.
- ❑ **PWM Control:** Fan speed is adjusted using PWM based on temperature.

9. Temp Monitor System using DHT and Motor



temp_monitor_system_dht_motor_1.ino

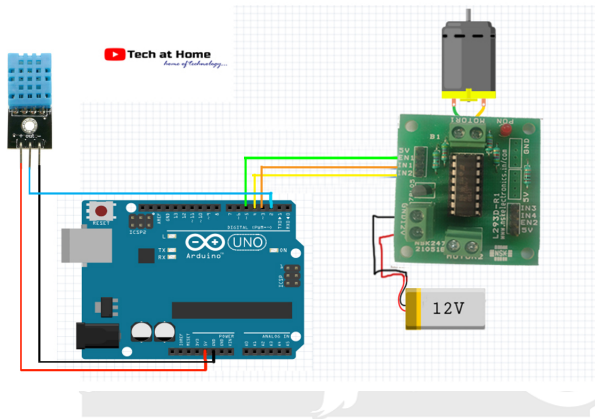
```
1  #include "DHT.h"
2  #define DHTPIN 2
3  #define DHTTYPE DHT11
4  DHT dht (DHTPIN, DHTTYPE);
5
6  int in1 = 2;
7  int in2 = 3;
8  int en1 = 6;
9  int y;
10
11 void setup() {
12     Serial.begin(9600);
13     dht.begin();
14
15     pinMode(in1,OUTPUT);
16     pinMode(in2,OUTPUT);
17     pinMode(en1,OUTPUT);
18 }
19
```

temp_monitor_system_dht_motor_1.ino

```
20 void loop() {
21     delay(2000);
22     float t = dht.readTemperature();
23     //checking if sensor is reading inputs
24     if (isnan(t)){
25         Serial.println("Failed to read from DHT");
26         return;
27     }
28
29     if (t <= 28){
30         digitalWrite(in1,LOW);
31         digitalWrite(in2,LOW);
32         analogWrite(en1,0);
33     }
34
35     else if ((t > 28) && (t <= 30)){
36         digitalWrite(in1,HIGH);
37         digitalWrite(in2,LOW);
38         analogWrite(en1,100);
39     }
40
41     else if (t > 30){
42         digitalWrite(in1,HIGH);
43         digitalWrite(in2,HIGH);
44         analogWrite(en1,255);
45     }
46 }
```

- without using map function. Fan speed is operated by the conditions shown in the code.

9. Temp Monitor System using DHT and Motor



10. Bluetooth Controlled Car



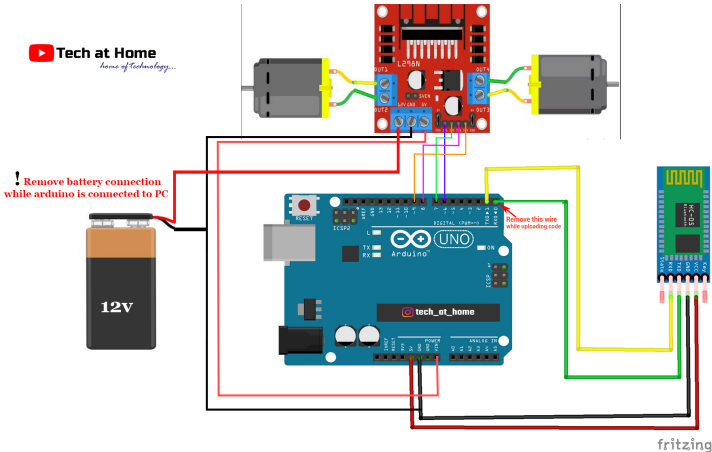
btmodule3_automated_car_l298n.ino

```
1  int in1 = 6;
2  int in2 = 7;
3  int in3 = 8;
4  int in4 = 9;
5  int data;
6
7  void setup() {
8      Serial.begin(9600);
9      pinMode(in1,OUTPUT);
10     pinMode(in2,OUTPUT);
11     pinMode(in3,OUTPUT);
12     pinMode(in4,OUTPUT);
13 }
14
15 void loop() {
16     while (serial.available() > 0){
17         data = serial.read();
18         Serial.println(data);
19     }
20
21     if (data == "F"){
22         digitalWrite(in1, HIGH);
23         digitalWrite(in2, LOW);
24         digitalWrite(in1, LOW);
25         digitalWrite(in2, HIGH);
26 }
```



```
27
28
29     if (data == "B"){
30         digitalWrite(in1, LOW);
31         digitalWrite(in2, HIGH);
32         digitalWrite(in1, LOW);
33         digitalWrite(in2, HIGH);
34     }
35
36     if (data == "L"){
37         digitalWrite(in1, HIGH);
38         digitalWrite(in2, LOW);
39         digitalWrite(in1, HIGH);
40         digitalWrite(in2, LOW);
41     }
42
43     if (data == "R"){
44         digitalWrite(in1, LOW);
45         digitalWrite(in2, HIGH);
46         digitalWrite(in1, LOW);
47         digitalWrite(in2, HIGH);
48     }
49 }
```


10. Bluetooth Controlled Car

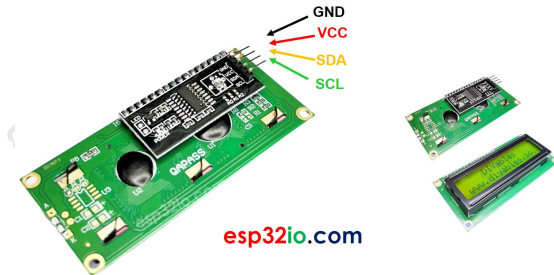


- Here I learned UART-based serial communication to receive and decode commands from the Bluetooth module.

1. DHT11 Sensor Values in LCD



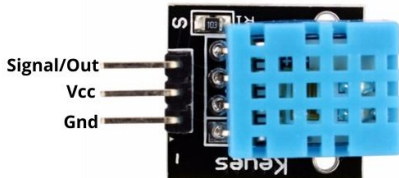
- ❑ LCD (Liquid Crystal Display) is a display module commonly used to show alphanumeric characters, symbols, and even custom patterns in embedded projects. The most popular LCD used with Arduino is the 16x2 LCD, which can display 2 lines with 16 characters each. For the experiment, we would be using the LCD with I2C interface.



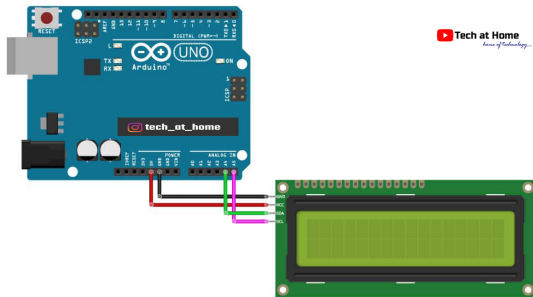
1. DHT11 Sensor Values in LCD



- DHT11 is a basic, low-cost digital sensor used to measure temperature and humidity. It contains a capacitive humidity sensor and a thermistor to measure the surrounding air and provides a digital signal on the data pin.



1. DHT11 Sensor Values in LCD

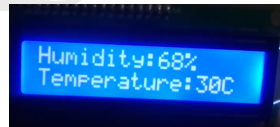


```
DHT dht(DHTPIN, DHTYPE);

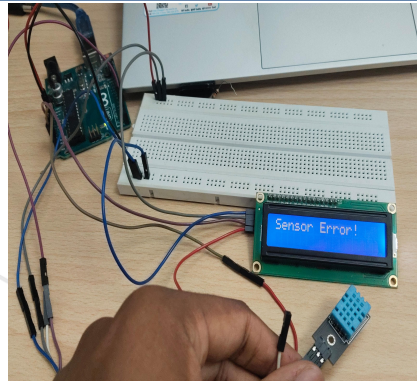
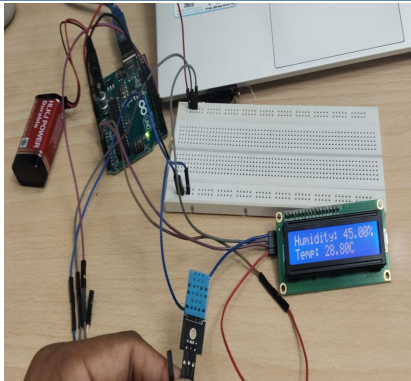
void setup() {
  Serial.begin(9600);
  dht.begin();
  lcd.begin();
  lcd.backlight();
  lcd.setBacklight(HIGH);
}

void loop() {
  int h = dht.readHumidity();
  int t = dht.readTemperature();
  int f = dht.readTemperature(true);
  //check if any reads fail and exit early (to try again)
  if (isnan(h)||isnan(t)||isnan(f)){
    Serial.println("Failed to read");
    return;
  }
  lcd.setCursor(0,0);
  lcd.print("Humidity:");
  lcd.print(h);
  lcd.print("%");
  delay(50);S
  //-----similarly temp
```

- ❑ For using LCD module we include libraries, "LiquidCrystal_I2C" and "Wire". The output looks like:



2. DHT Values in LCD



- ❑ Some sensors failed to sense the temperature and humidity as shown in fig 2. Hence LCD showed "Sensor Error!".
- ❑ Sometimes we have to provide external power as the power supply wasnot sufficient lead to garbage values in LCD.

2. Home Automation Project (Fans, Light)

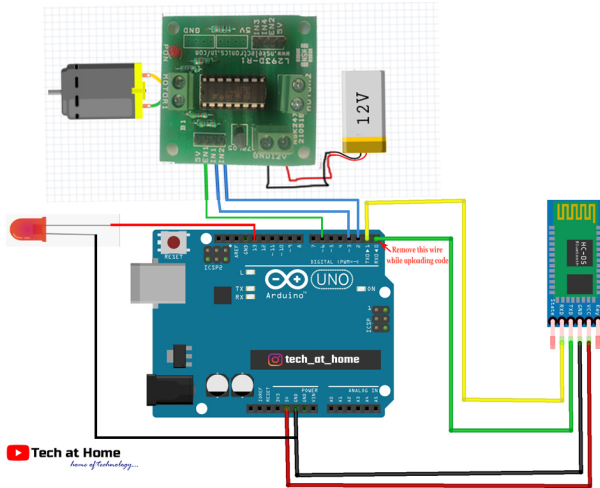


btmodule2_fanregulator_lightswitch.ino

```
1  int in1 = 2;
2  int in2 = 3;
3  int en1 = 6;
4  int led = 13;
5
6  void setup() {
7      Serial.begin(9600);
8      pinMode(led,OUTPUT);
9      pinMode(in1,OUTPUT);
10     pinMode(in2,OUTPUT);
11     pinMode(en1,OUTPUT);
12 }
13
14 void loop() {
15     while (serial.available() > 0){
16         data = serial.read();
17         Serial.println(data);
18     }
19 }
```

```
20  if (data == "A"){
21     digitalWrite(13,HIGH);
22 }
23
24  if (data == "B"){
25     digitalWrite(13,LOW);
26 }
27
28  if (data == "C"){
29     digitalWrite(in1, HIGH);
30     digitalWrite(in2, LOW);
31     analogWrite(en1, 0);
32 }
33
34  if (data == "D"){
35     digitalWrite(in1, HIGH);
36     digitalWrite(in2, LOW);
37     analogWrite(en1, 100);
38 }
39
40  if (data == "E"){
41     digitalWrite(in1, HIGH);
42     digitalWrite(in2, LOW);
43     analogWrite(en1, 255);
44 }
45 }
```

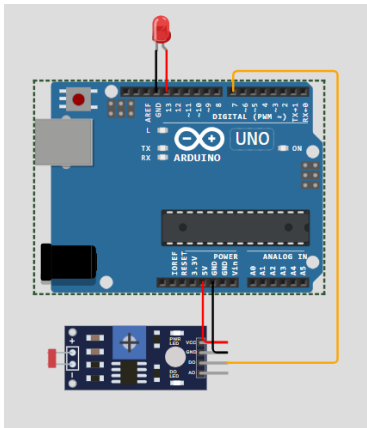
2. Home Automation Project (Fans, Light)



3. Smart Street Lamps



In this project, we built smart street lamps that during the day (when detecting sunlight) are switched off and switched on during the night.

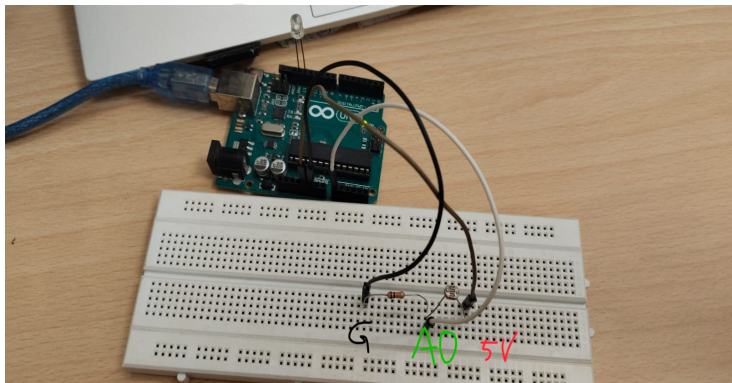


```
1 //automatic street lamp control using LDR sensors
2 int ldr = 7;
3 int led = 13;
4 int x;
5 void setup() {
6     Serial.begin(9600);
7     pinMode(ldr, INPUT);
8     pinMode(led, OUTPUT);
9 }
10
11 void loop() {
12     x = digitalRead(ldr);
13     Serial.println(x);
14
15     delay(200);
16
17     if (x == 1){
18         digitalWrite(led, LOW);
19     }
20     else{
21         digitalWrite(led, HIGH);
22     }
23 }
24
```


3. Automatic Street Lamps using LDR



In this project, we built smart street lamps that during the day (when detecting sunlight) are switched off and switched on during the night.



□ [Link for Video Demonstration in GitHub](#)

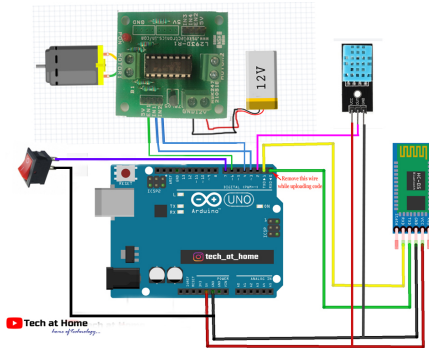
4. Motor Control with Auto and Manual Mode



```
1 //using dht, lcd, btmodule, toggle 30 void loop() {
2 #include "DHT.h" 31 Mode = digitalRead(MODE_SW);
3 #include <liquidCrystal_I2C.h> 32
4 #include <Wire.h> 33 if (Mode == HIGH){ //Manual mode
5 #define DHTPIN 2 34 lcd.setCursor(0,0);
6 #define DHTTYPE DHT11 35 lcd.print("Mode:Manual");
7 DHT dht (DHTPIN, DHTTYPE); 36 while (Serial.available()>0){
8 LiquidCrystal_I2C lcd (0x27,16,2); 37 data = Serial.read();
9 38 if (data == "C"){
10 int in1 = 3; 39 lcd.setCursor(0,1);
11 int in2 = 4; 40 lcd.print("Fan:OFF");
12 int en = 6; 41 delay(10);
13 int MODE_SW = 7; 42 digitalWrite(in1, HIGH);
14 int data; 43 digitalWrite(in2, LOW);
15 int Mode; 44 analogWrite(en, 0);
16 45 }
17 void setup() { 46 if (data == "D"){
18 Serial.begin(9600); 47 lcd.setCursor(0,1);
19 dht.begin(); 48 lcd.print("Fan:LOW");
20 lcd.begin(16,2); 49 delay(10);
21 lcd.backlight(); 50 digitalWrite(in1, HIGH);
22 lcd.setBacklight(HIGH); 51 digitalWrite(in2, LOW);
23 52 analogWrite(en, 100);
24 pinMode(in1, OUTPUT); 53 }
25 pinMode(in2, OUTPUT); 54 if (data == "E"){
26 pinMode(en, OUTPUT); 55 lcd.setCursor(0,1);
27 pinMode(MODE_SW, INPUT); 56 lcd.print("Fan:HIGH");
28 57 delay(10);
29 58 digitalWrite(in1, HIGH);
30 59 digitalWrite(in2, LOW);
31 60 analogWrite(en, 255);
32 61 }
33 62 }
34 63 }
35 64 }
36 65 }
37 66 }
38 67 }
39 68 }
40 69 }
41 70 }
42 71 }
43 72 }
44 73 }
45 74 }
46 75 }
47 76 }
48 77 }
49 78 }
50 79 }
51 80 }
52 81 }
53 82 }
54 83 }
55 84 }
56 85 }
57 86 }
58 87 }
59 88 }
60 89 }
61 90 }
62 91 }
63 92 }
64 93 }
65 94 }
66 95 }
67 96 }
68 97 }
69 98 }
70 99 }
```

2nd image shows manual mode using "BT module" and 3rd image shows automatic mode using "DHT11 sensor".

4. Motor Control with Auto and Manual Mode



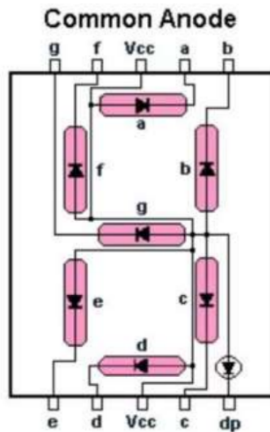
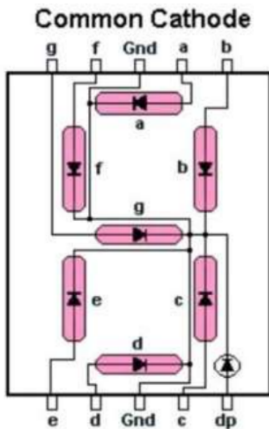
- ☐ Toggle switch is used to switch between auto and manual mode.
- ☐ The results get displayed using the LCD monitor.
- ☐ We can also modify the auto mode to make use of the map function to control the speed of fan according to each temp value by DHT11.

5. Traffic Light Control and 7 Segment Display



Seven Segment Display

- Two types: Common Cathode (CC) and Common Anode (CA).
- The difference is the polarity of the LEDs and common terminal.



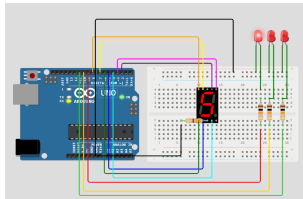
5. Traffic Light Control and 7 Segment Display



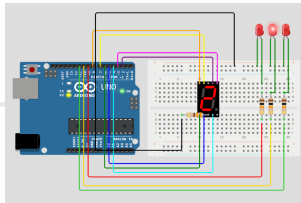
```
1  #include "SevSeg.h"
2  SevSeg S;
3  byte CommonPins[] = {};
4  byte SegPins[] = {2,3,4,5,6,7,8};
5  int red =9;
6  int yellow = 10;
7  int green = 11;
8
9  void setup() {
10     S.begin(COMMON_CATHODE, 1, CommonPins, SegPins, 1);
11     pinMode(9, OUTPUT);
12     pinMode(10, OUTPUT);
13     pinMode(11, OUTPUT);
14
15     digitalWrite(red, HIGH); //it begins with red on
16 }
17
18 void loop() {
19     for(int i = 9; i>=0; i--){
20         display(i); //displays number
21         delay(2000);
22         if(i <= 3){
23             digitalWrite(yellow, HIGH);
24             digitalWrite(red, LOW);
25         }
26
27
28         digitalWrite(green , HIGH);
29         digitalWrite(yellow, LOW);
30
31         for(int i = 9; i>=0; i--){
32             display(i); //displays number
33             delay(2000);
34             if(i <= 3){
35                 digitalWrite(yellow, HIGH);
36                 digitalWrite(green, LOW);
37             }
38         }
39
40         digitalWrite(red, HIGH);
41         digitalWrite(yellow, LOW);
42         digitalWrite(green, LOW);
43     }
44
45     void display(int num) {
46         S.setNumber(num);
47         S.refreshDisplay();
48         delay(20);
49     }
50 }
```

- ❑ Here I used the "SevSeg" library to display numbers on 7 segment displays by handling segment control automatically.

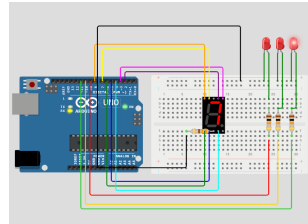
5. Traffic Light Control and 7 Segment Display



Red Light Phase



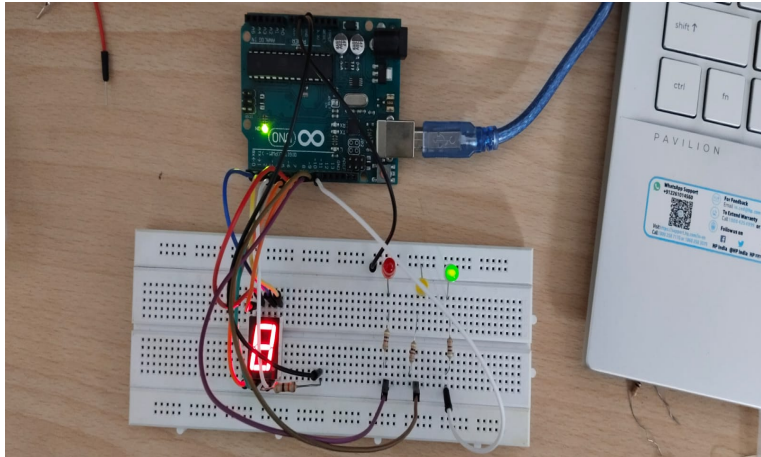
Yellow Light Phase



Green Light Phase

[Click here to view the simulation video](#)

6. Traffic Light Control and 7 Segment Display



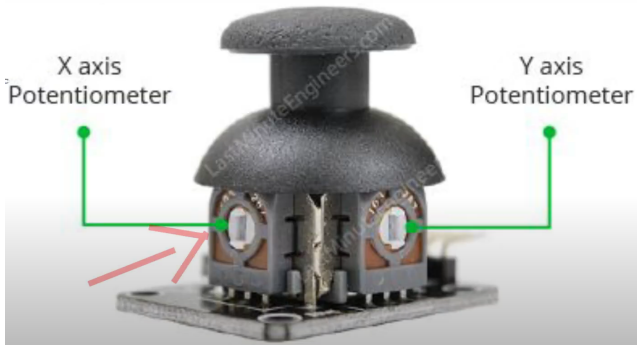
□ [Link for Simulation and Video Demonstration in GitHub](#)

6. Controlling lights with Joystick Module



Joystick Module

- ❑ Important component in game controllers
- ❑ Has two axes, one potentiometer mounted separately for each axes.



6. Controlling lights with Joystick Module



Joystick_LED.ino

```
1  int x_pin = A0; //Joystick X axis
2  int y_pin = A1; //Joystick y axis
3  int sw_pin = 2;
4  int led_F = 3;
5  int led_B = 5;
6  int led_R = 6;
7  int led_L = 9;
8
9  void setup(){
10     Serial.begin(9600);
11     pinMode(A0, INPUT);
12     pinMode(A1, INPUT);
13     pinMode(2, INPUT);
14     digitalWrite(2, HIGH);
15     pinMode(3, OUTPUT);
16     pinMode(5, OUTPUT);
17     pinMode(6, OUTPUT);
18     pinMode(9, OUTPUT);
19 }
20 void loop(){
21     int x_data = analogRead(A0);
22     int y_data = analogRead(A1);
23     int sw_data = digitalRead(2);
24     Serial.print("x_data:");
25     Serial.print(x_data);
26     Serial.print("\t");
27     Serial.print("y_data:");
28     Serial.print(y_data);
29     Serial.print("\t");
30     Serial.print("sw_data:");
```

```
if(y_data <= 500){ //Forward LED
    int brightness_F = map(y_data, 500, 0, 0, 255);
    analogWrite(led_F, brightness_F);
}

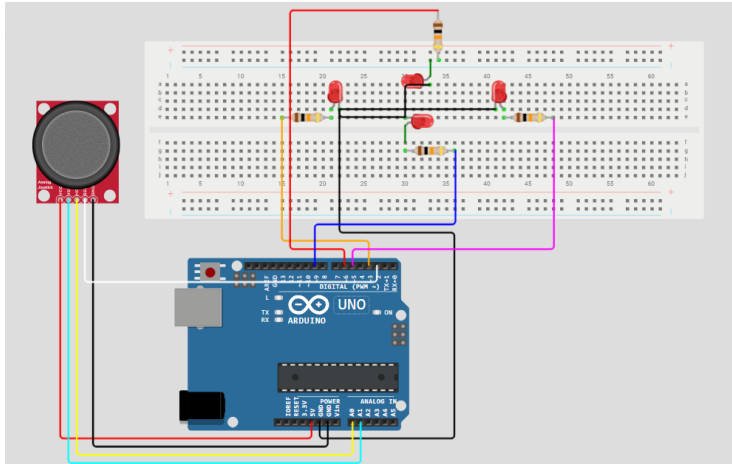
else if(y_data >= 550){ //Backward LED
    int brightness_B = map(y_data, 550, 1022, 0, 255);
    analogWrite(led_B, brightness_B);
}

else if(x_data >= 550){ //Right LED
    int brightness_R = map(x_data, 550, 1022, 0, 255);
    analogWrite(led_R, brightness_R);
}

else if(x_data <= 500){ //Left LED
    int brightness_L = map(x_data, 500, 0, 0, 255);
    analogWrite(led_L, brightness_L);
}

else{
    digitalWrite(led_F, LOW);
    digitalWrite(led_B, LOW);
    digitalWrite(led_R, LOW);
    digitalWrite(led_L, LOW);
}
```

6. Controlling lights with Joystick Module



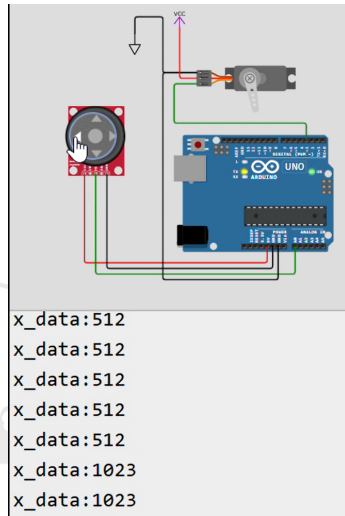
7. Controlling Servo Motor using Joystick



Controlling_Servomotor_with_Joystick.ino

diagram.json

```
1  #include<Servo.h>
2  int x_pin = A0;
3  Servo s1;
4  int pos = 90;
5  void setup() {
6    Serial.begin(9600);
7    pinMode(A0, INPUT);
8    s1.attach(3);
9    s1.write(pos); //start with 90 deg
10 }
11
12 void loop() {
13   int x_data = analogRead(A0);
14   Serial.print("x_data:");
15   Serial.println(x_data);
16   if (x_data >= 550){
17     if(pos <= 180){
18       pos = pos + 10;
19       s1.write(pos);
20     }
21   }
22   else if (x_data <= 500){
23     if(pos >= 0){
24       pos = pos - 10;
25       s1.write(pos);
26     }
27   }
28 }
```



Simulation Link

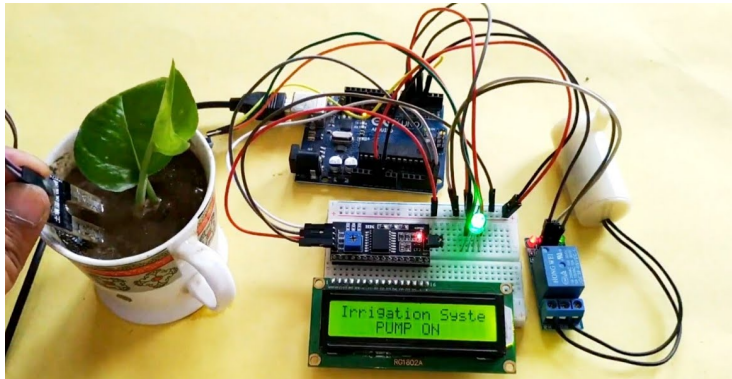
8. Smart Irrigation System



Smart_Irrigation_System.ino

```
1  #include<LiquidCrystal_I2C.h>
2  LiquidCrystal_I2C lcd(0x27,16,2);
3
4  int sensor_pin = A0;
5  int relay_pin = 7;
6
7  void setup(){
8      Serial.begin(9600);
9      lcd.begin(16,2);
10     lcd.backlight();
11     lcd.setBacklight(HIGH);
12     pinMode(sensor_pin, INPUT);
13     pinMode(relay_pin, OUTPUT);
14 }
15
16 void loop(){
17     int sensor_data = analogRead(sensor_pin);
18     Serial.print("Sensor_data:");
19     Serial.print(sensor_data);
20     Serial.print("\t | ");
21
22     if(sensor_data > 950){
23         Serial.println("No moisture, Soil is dry");
24         digitalWrite(relay_pin, HIGH);
25         lcd.setCursor(0,0);
26         lcd.print("Soil Dry");
27         lcd.setCursor(0,1);
28         lcd.print("Motor ON");
29     }
30     else if(sensor_data >= 400 && sensor_data <= 950){
31         Serial.println("There is some moisture, Soil is medium");
32         digitalWrite(relay_pin, LOW);
33         lcd.setCursor(0,0);
34         lcd.print("Soil Medium");
35         lcd.setCursor(0,1);
36         lcd.print("Motor OFF");
37     }
38     else if(sensor_data < 400){
39         Serial.println("Soil is wet");
40         digitalWrite(relay_pin, LOW);
41         lcd.setCursor(0,0);
42         lcd.print("Soil Wet");
43         lcd.setCursor(0,1);
44         lcd.print("Motor OFF");
45     }
46     delay(100);
47 }
```

8. Smart Irrigation System



- ❑ Soil moisture sensor measures the moisture and waters the crops accordingly which are being shown in lcd.

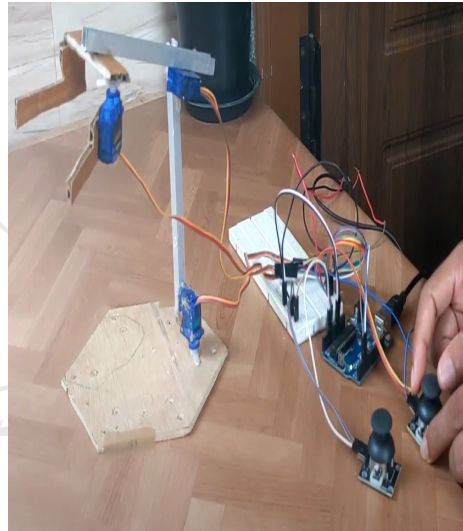
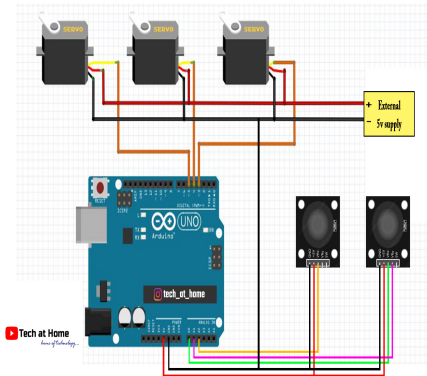
9. Joystick Controlled Robotic Arm



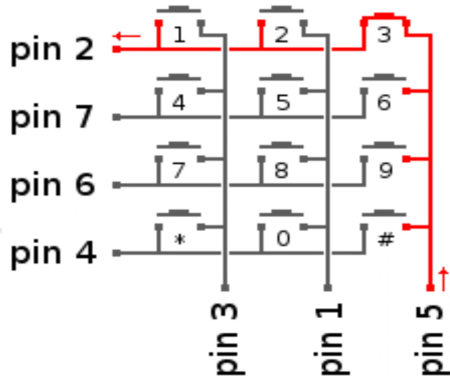
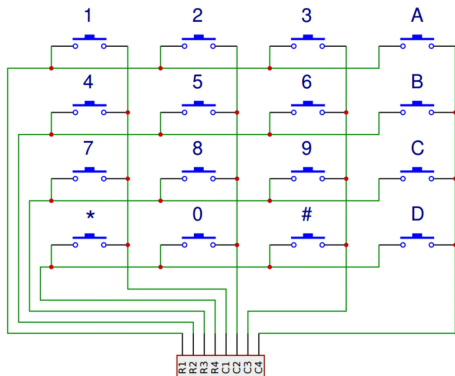
ARM.ino

```
1 #include<Servo.h>
2 int x_pin1 = A0; //Joystick-1(X and Y axis)
3 int y_pin1 = A1; //Joystick-2(X axis)
4 int x_pin2 = A2;
5 int pos1, pos2, pos3 = 0; //Initialize with 0 deg
6 Servo s1;
7 Servo s2;
8 Servo s3;
9
10 void setup(){
11     Serial.begin(9600);
12     pinMode(A0, INPUT);
13     pinMode(A1, INPUT);
14     pinMode(A2, INPUT);
15     s1.attach(3); //Servo motor attached to pin 3
16     s1.write(pos1);
17     s2.attach(4); //Servo motor attached to pin 4
18     s2.write(pos2);
19     s3.attach(5); //Servo motor attached to pin 5
20     s3.write(pos3);
21 }
22
23 void loop(){
24     //Reading X and Y axis of joystick-1
25     int x_data1 = analogRead(A0);
26     int y_data1 = analogRead(A1);
27     //Reading X axis of joystick-2
28
29     Serial.print("x_data1:");
30     Serial.print(x_data1);
31     Serial.print("\t");
32     Serial.print("y_data1:");
33     Serial.print(y_data1);
34     Serial.print("\t");
35     Serial.print("x_data2:");
36     Serial.print(x_data2);
37     delay(20);
38     //Servo 1 control with joystick-1
39     if(x_data1 >= 550){
40         if(pos1 <= 180){
41             pos1 = pos1 + 3;
42             s1.write(pos1);
43         }
44     }
45     else if(x_data1 <= 450){
46         if(pos1 >= 0){
47             pos1 = pos1 - 3;
48             s1.write(pos1);
49         }
50     }
51     //Servo 2 control with joystick-1
52     if(y_data1 >= 550){
53         if(pos2 <= 180){
54             pos2 = pos2 + 2;
55             s2.write(pos2);
56         }
57     }
58     else if(y_data1 <= 450){
59         if(pos2 >= 0){
60             pos2 = pos2 - 2;
61             s2.write(pos2);
62         }
63     }
64
65     //Servo 3 control with joystick-2
66     if(x_data2 >= 550){
67         if(pos3 <= 180){
68             pos3 = pos3 + 3;
69             s3.write(pos3);
70         }
71     }
72     else if(x_data2 <= 450){
73         if(pos3 >= 0){
74             pos3 = pos3 - 3;
75             s3.write(pos3);
76         }
77     }
78 }
79
80
```

9. Joystick Controlled Robotic Arm



10. 4x4 Keypad Module with LCD



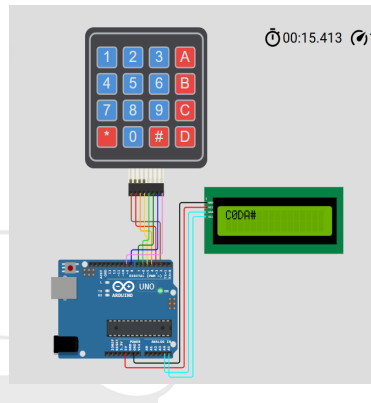
- Internal structure of a keyboard module contains multiple push buttons are connected in a matrix format where pressing one button connects one row and a column.

10. 4x4 Keypad Module with LCD



Arduino_keypad_with_LCD_module.ino diagram.json

```
1  #include <Keypad.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <Wire.h>
4  LiquidCrystal_I2C lcd(0x27,16,2);
5  int i;
6  const char number_of_rows = 4;
7  const char number_of_columns = 4;
8
9  char row_pins[number_of_rows] = {2,3,4,5};
10 char column_pins[number_of_columns] = {6,7,8,9};
11 char key_array[number_of_rows][number_of_columns]= {
12   {'1','2','3','A'},
13   {'4','5','6','B'},
14   {'7','8','9','C'},
15   {'*','0','#','D'}};
16 };
17
18 Keypad k = Keypad(makeKeymap(key_array),row_pins,column_pins,number_of_rows,number_of_columns);
19
20 void setup() {
21   Serial.begin(9600);
22   lcd.begin(16,2);
23   lcd.backlight();
24   lcd.setBacklight(HIGH);
25 }
26
27 void loop() {
28   char key_pressed = k.getKey();
29   if (key_pressed){
30     Serial.println(key_pressed);
31     lcd.setCursor(i,0);
32     lcd.print(key_pressed);
33     i = i+1;
34   }
35 }
```



1. Smart Door Lock System



Smart_Doorlock_using_KeypadModule.ino diagram.json

```
1 #include <Keypad.h>
2 #include <Servo.h>
3 #include <LiquidCrystal_I2C.h>
4 #include <Wire.h>
5 LiquidCrystal_I2C lcd(0x27,16,2);
6 Servo s1;
7
8 const int Actual_Pin_size = 4;
9 const int Password_size = (Actual_Pin_size + 1);
10 char password[Password_size] = "12AB"; //set password
11 char Entered_pin[Password_size];
12
13 const char no_of_rows =4;
14 const char no_of_columns = 4;
15 byte row_pins[] = {2,3,4,5};
16 byte column_pins[] = {6,7,8,9};
17 char key_array[no_of_rows][no_of_columns] = {
18   {'1','2','3','A'},
19   {'4','5','6','B'},
20   {'7','8','9','C'},
21   {'*','0','#','D'}
22 };
23 Keypad k = Keypad(makeKeymap(key_array),row_pins,column_pins,no_of_rows,no_of_columns);
24
25 int Red = 12;
26 int Green = 13;
27 char key;
28 int i = 0;
29 int lock_status = 1; //initially locked
30
31 void setup() {
32   Serial.begin(9600);
33   lcd.begin(16,2);
34   lcd.backlight();
35   lcd.setBacklight(HIGH);
```

Smart_Doorlock_using_KeypadModule.ino diagram.json

```
37 pinMode(Red, OUTPUT);
38 pinMode(Green, OUTPUT);
39 digitalWrite(Red, HIGH);
40 digitalWrite(Green, LOW);
41 s1.attach(10);
42 s1.write(0);
43
44 lcd.setCursor(4,0);
45 lcd.print("Welcome!");
46 delay(2000);
47 lcd.setCursor(1,0);
48 lcd.print("Door: Locked");
49 delay(2000);
50 lcd.clear();
51 }
52
53 void loop() {
54   if (lock_status == 1) { //Trigger unlock if door locked
55     Unlock_activate();
56   }
57   if (lock_status == 0) { //Trigger lock if door unlocked
58     lcd.setCursor(0,0);
59     lcd.print("Enter * to lock:");
60
61     key = k.getKey();
62     if(key){
63       //Serial.println(key);
64       if (key == '*'){ //Use * specifically ""
65         Lock_activate();
66         lcd.clear();
67         lcd.setCursor(0,0);
68         lcd.print("Door Locked");
69         delay(2000);
70         lcd.clear();
71         lock_status = 1;
```

1. Smart Door Lock System



Smart_Doorlock_using_KeypadModule.ino

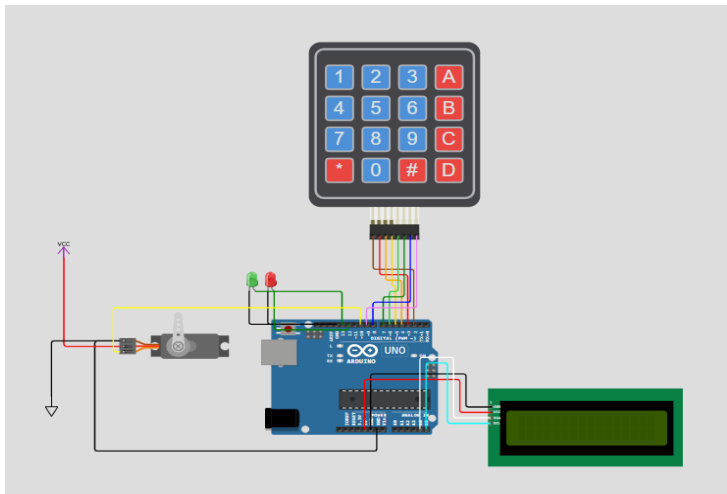
diagram.json

```
77 void Unlock_activate(){
78     lcd.setCursor(0,0);
79     lcd.print("Enter Pin:");
80
81     key = k.getKey();
82     if(key){
83         Entered_pin[i] = key;
84         lcd.setCursor(i,1);
85         lcd.print("*");
86         i++;
87
88         if(i == (Password_size - 1)){
89             // Serial.println(Entered_pin);
90             Entered_pin[i] = '\0';
91             delay(1000);
92             if(strcmp(Entered_pin,password)== 0){
93                 digitalWrite(Green, HIGH);
94                 digitalWrite(Red, LOW);
95                 s1.write(90);
96
97                 lcd.clear();
98                 lcd.setCursor(0,0);
99                 lcd.print("Door: Unlocked");
100
101                 i = 0;
102                 delay(1000);
103                 lock_status = 0;
104             }
```

```
else{
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Wrong Pin");
    delay(2000);
    lcd.clear();
    i = 0;
    lock_status = 1; // Setting the lock to 1
}
```

```
void Lock_activate(){
    digitalWrite(Red, HIGH);
    digitalWrite(Green, LOW);
    s1.write(0);
    delay(1000);
}
```

1. Smart Door Lock System



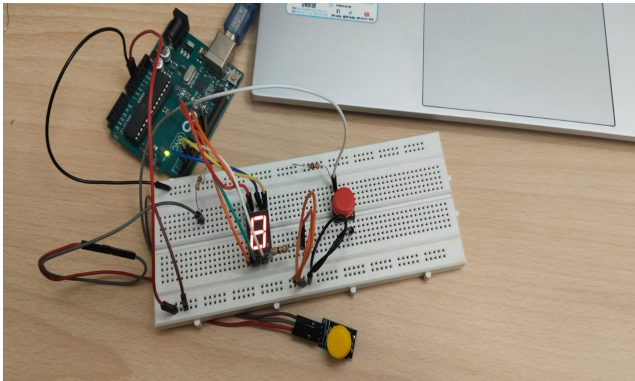
□ [Link for Simulation Video in GitHub](#)

5. ++ and -- Counter using Pushbuttons



```
1  #include "SevSeg.h"
2  SevSeg S;
3  byte CommonPins[] = {};           // common pin numb
4  byte SegPins[] = {2,3,4,5,6,7,8}; // 7-segment displ
5
6  int btn1=9; //button for increment
7  int btn2=10; //button for decrement
8  int cnt=0;
9  int incPrev, decPrev;
10
11 void setup() {
12     S.begin(COMMON_ANODE, 1, CommonPins, SegPins, 1);
13     pinMode(btn1, INPUT_PULLUP); // assumes active LOW
14     pinMode(btn2, INPUT_PULLUP);
15 }
16
17 void loop(){
18     int inc = digitalRead(9);
19     int dec = digitalRead(10);
20
21     //Increment
22     if((inc == HIGH) && (cnt < 9) && (inc != incPrev )){
23         delay(100);
24         cnt++;
25     }
26
27     //Decrement
28     if((dec == HIGH) && (cnt > 0) && (dec != decPrev)){
29         delay(100);
30         cnt--;
31     }
32
33     //Logic to print digit/character on 7 segment display
34     S.setNumber(cnt);
35     S.refreshDisplay();
36     delay(100);
37
38     //storing the button states
39     incPrev = inc;
40     decPrev = dec;
41 }
42
```

5. ++ and -- Counter using Pushbuttons



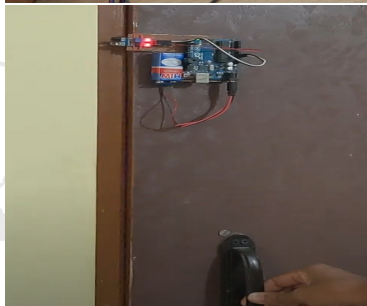
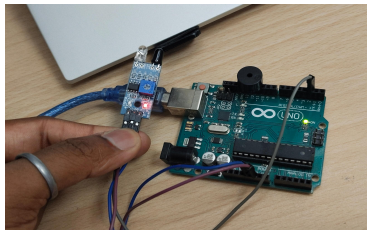
☐ [Link for Video Demonstration in GitHub](#)

7. Arduino Door Security System using IR Sensor



Arduino_Door_Security_System_using_IRSensor.ino

```
1  byte buzzer = 11;
2  byte ir_sensor = 2;
3
4  void setup() {
5      Serial.begin(9600);
6      // put your setup code here, to run once:
7      pinMode(ir_sensor, INPUT);
8      pinMode(buzzer, OUTPUT);
9  }
10
11 void loop() {
12     // put your main code here, to run repeatedly:
13     int sensor_state = digitalRead(ir_sensor);
14     Serial.println(sensor_state);
15     if (sensor_state == LOW){
16         analogWrite(buzzer, 200);
17         delay(110);
18         analogWrite(buzzer, 100);
19         delay(110);
20     }
21
22     else{
23         digitalWrite(buzzer, HIGH);
24         delay(110);
25     }
26 }
27
```



Link for Video Demonstration in GitHub



❑ What is Learning?

Learning is the act of acquiring knowledge over experience to improve performance.

❑ Machine Learning

ML is the branch of data science that enables computer systems to learn patterns and make predictions from the data without being explicitly programmed for each specific task.

Machine learning problems broadly divided into:

1. Supervised learning
2. Unsupervised learning



□ Supervised Learning

A supervised learning algorithm analyzes the previously labeled data and produces an inferred function, which can be used for mapping new instances.

Let $(\vec{X}_i, y_i), \forall i = 1, 2, \dots, n$, be the instances in a dataset, where $\vec{X}_i = [x_{i1}, x_{i2}, \dots, x_{im}] \in \mathbb{R}^m$ is an m -dimensional feature vector and n is the number of instances (also called data points). Here, $x_{ij}, \forall j = 1, 2, \dots, m$, are the individual features of the i^{th} data point, and y_i is the corresponding target variable for $\vec{X}_i, \forall i$. The aim is to build a model using this dataset to estimate the target variable of a new instance, say, $\vec{X}_0 \in \mathbb{R}^m$.

□ Broadly classified into *Classification* and *Regression* problems.

Supervised & Unsupervised Learning

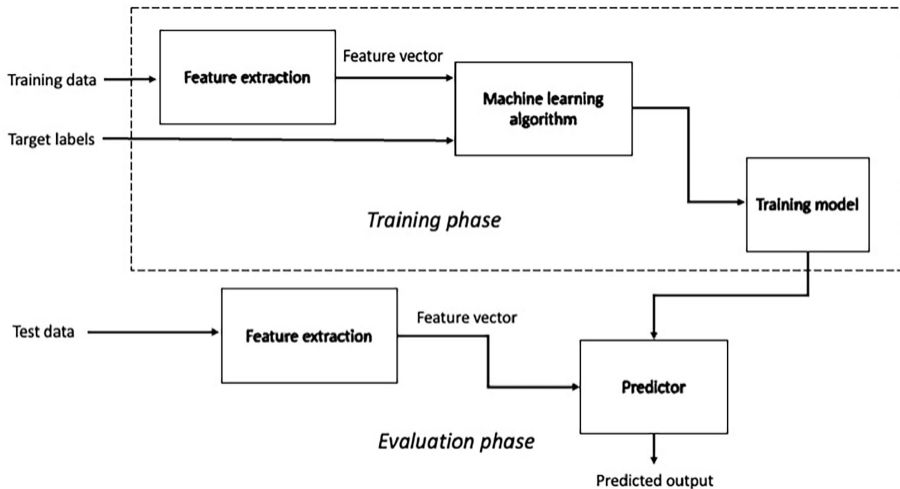


Figure 1.1: Block diagram of supervised learning approach

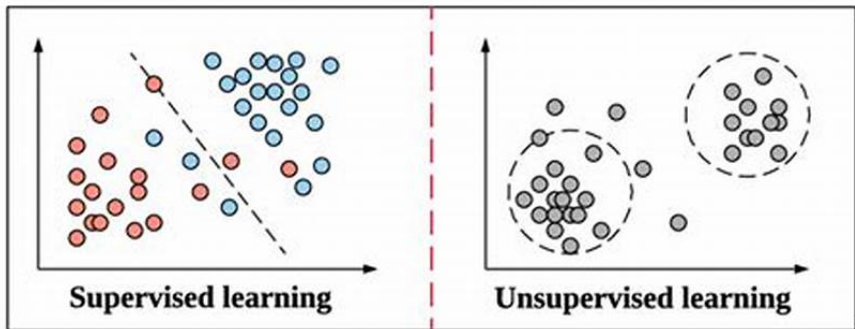
Supervised & Unsupervised Learning



□ Unsupervised Learning

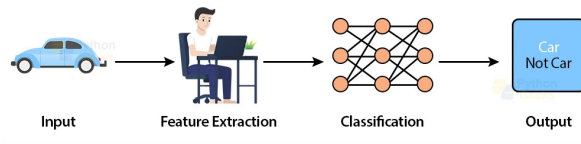
In the unsupervised learning, no class labels are available in the given data to build the model. The objective is to find natural groups or clusters of similar objects within the data.

- Broadly divided to *Clustering* and *Density Estimation* problems.

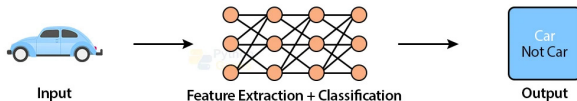


- ❑ ML algorithms cannot learn directly from raw unstructured data.
- ❑ DL algorithms can directly take raw data as input and can extract the relevant features automatically, therefore bypassing the need for manual feature extraction.

Machine Learning



Deep Learning



- ❑ Computationally expensive, Needs (**GPU**) or (**TPU**).



- ❑ The layered structure of Artificial Neural Network (ANN) is inspired by the biological nervous system.
- ❑ An ANN comprises of neurons or nodes in a layered structure where each layer is connected to another layer to analyze complex patterns and relationships in data.

Q. How does an ANN work ?

❑ Forward Propagation:

- ❑ Inputs are multiplied by weights and summed.
- ❑ The result is passed through activation functions layer by layer.

❑ Loss Calculation:

- ❑ The predicted output is compared with the actual value.
- ❑ A *loss function* (MSE, cross-entropy) is used to quantify the error.

❑ Backpropagation:

- ❑ Computes gradients of the loss with respect to each weight.
- ❑ Updates weights using *gradient descent* to minimize the loss.

Artificial Neural Network (ANN)



❑ Building Sequential Model by adding layers through ".add()"

```
from tensorflow.keras.layers import Dense
model = Sequential()
model.add(Dense(512, activation = 'relu', input_shape = (784,)))
model.add(Dense(128, activation = 'relu'))
model.add(Dense(num_class, activation = 'softmax'))
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

[18]

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 512)	401,920
dense_13 (Dense)	(None, 128)	65,664
dense_14 (Dense)	(None, 10)	1,290

Total params: 468,874 (1.79 MB)

Trainable params: 468,874 (1.79 MB)

Non-trainable params: 0 (0.00 B)

Artificial Neural Network (ANN)



- Training the model & Finding accuracy on test data.

```
model.fit(X_train, y_train, batch_size=128, epochs=10, verbose=1)
```

```
Epoch 1/10
469/469 ————— 3s 5ms/step - accuracy: 0.8751 - loss: 0.4318
Epoch 2/10
469/469 ————— 2s 5ms/step - accuracy: 0.9732 - loss: 0.0897
Epoch 3/10
469/469 ————— 2s 5ms/step - accuracy: 0.9822 - loss: 0.0565
Epoch 4/10
469/469 ————— 2s 5ms/step - accuracy: 0.9879 - loss: 0.0391
Epoch 5/10
469/469 ————— 2s 5ms/step - accuracy: 0.9916 - loss: 0.0279
Epoch 6/10
469/469 ————— 2s 5ms/step - accuracy: 0.9944 - loss: 0.0177
Epoch 7/10
469/469 ————— 3s 5ms/step - accuracy: 0.9956 - loss: 0.0145
```

```
score = model.evaluate(X_test, y_test)
print('loss on test data: ', score[0])
print('accuracy on test data:', score[1])
```

```
313/313 ————— 1s 3ms/step - accuracy: 0.9784 - loss: 0.0932
loss on test data: 0.07661768794059753
accuracy on test data: 0.9821000099182129
```



What is a CNN?

- ☐ A type of deep learning model specifically designed to process grid-like data such as images.
- ☐ Mimics the visual perception mechanism of the human brain.
- ☐ Automatically learns spatial hierarchies of features through layers.
- ☐ Commonly used in image classification, object detection, and medical image analysis.

Why CNN for Images?

- ☐ Preserves spatial structure of the data.
- ☐ Reduces the number of parameters compared to fully connected networks.
- ☐ Learns relevant features automatically using filters.

Convolutional Neural Network (CNN)



```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dropout, Dense

model = Sequential()
model.add(Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape = (28,28,1)))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Conv2D(64, kernel_size=(3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(num_class, activation='softmax'))
model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_5 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_2 (Flatten)	(None, 1600)	0
dropout_2 (Dropout)	(None, 1600)	0
dense_2 (Dense)	(None, 10)	16,010



Model Layers and Their Functions

- ❑ **Input Layer:** Accepts input images.
- ❑ **Conv2D (32 filters, 3×3):** Applies convolution to input locally to extract features.
- ❑ **MaxPooling2D (2×2):** Reduces the spatial dimensions (downsamples) while keeping important features.
- ❑ **Flatten:** Converts the 2D feature maps into a 1D vector for feeding into the dense layer.
- ❑ **Dropout (rate = 0.5):** Randomly deactivates 50% of neurons during training to prevent overfitting.
- ❑ **Dense (Softmax):** Outputs probabilities for each class using the softmax activation function.

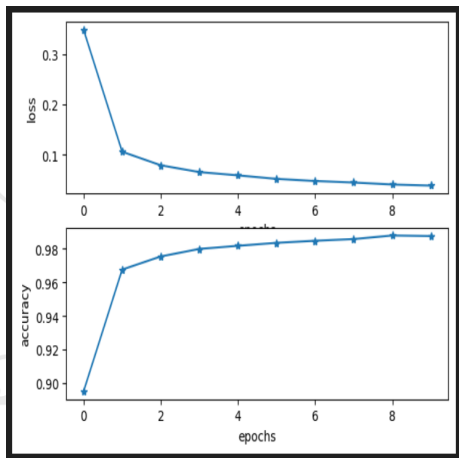
Convolutional Neural Network (CNN)



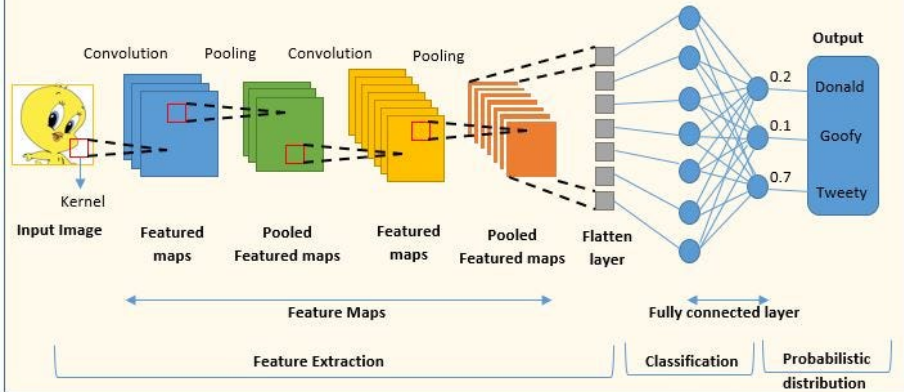
```
score = model.evaluate(X_test, y_test)
print('loss on test data: ', score[0])
print('accuracy on test data:', score[1])
```

313/313 ————— 2s 5ms/step - accuracy: 0.9904 - loss: 0.0309
loss on test data: 0.025411060079932213
accuracy on test data: 0.9922999739646912

- ❑ CNN achieves 99% test accuracy, slightly outperforming the 98% from the feedforward ANN on the MNIST dataset.
- ❑ Image in the right shows the plot of loss and accuracy w.r.t. epochs during training of CNN respectively.



A Typical Convolutional Neural Network (CNN)



Q. Why TinyML?

- ❑ Large deep learning algorithms typically run on powerful desktops, data centers, or cloud servers.
- ❑ Smaller edge devices and microcontrollers are severely resource-constrained (e.g., 256KB memory and 64MHz CPU).
- ❑ TinyML brings ML inference to low-power, low-memory devices.
- ❑ Enables real-time, offline, and privacy-preserving applications on the edge.



TensorFlow Lite

- ❑ With TensorFlow Lite Converter, we can convert a TensorFlow model into a compressed **FlatBuffers** format that is optimized for mobile and edge devices.



After we make an NN model, we save the baseline model by:

```
model.save('baseline_model.h5')
```

Converting to TFLite:

```
from tensorflow.keras.models import load_model
baseline_model = load_model('baseline_model.h5')

converter = tf.lite.TFLiteConverter.from_keras_model(baseline_model)
tflite_model = converter.convert()
```

Export model as a concrete function:

```
func = tf.function(baseline_model).get_concrete_function(tf.TensorSpec(baseline_model.inputs[0].shape, baseline_model.inputs[0].dtype))
func.graph.as_graph_def() # serialized graph representation of the concrete function

converter = tf.lite.TFLiteConverter.from_concrete_functions([func]) # converting the concrete function to TFLite
tflite_model = converter.convert()
```

Applying the TFLite model by ".invoke()":

```
tflite_model_file = 'tflite_models/model.tflite'
interpreter = tf.lite.Interpreter(model_path = str(tflite_model_file))

interpreter.allocate_tensors()

input_index = interpreter.get_input_details()[0]['index']
output_index = interpreter.get_output_details()[0]['index']

pred_list = []
for images in X_test:
    input_data = np.array(images, dtype=np.float32)
    input_data = input_data.reshape(1, input_data.shape[0], input_data.shape[1], 1)

    interpreter.set_tensor(input_index, input_data)
    interpreter.invoke()      #applies the tflite model

    prediction = interpreter.get_tensor(output_index)
    prediction = np.argmax(prediction)
    pred_list.append(prediction)
```



- ❑ First, we create an instance of the Interpreter class. It takes the path containing the .TFLITE file as an input.
- ❑ Allocate memory to the Interpreter by calling the function `allocate_tensors()`.
- ❑ Use `details()` to inspect input and output tensors.
- ❑ Serialization converts the model graph into a portable format (e.g., .pb file) that can be:
 - ❑ Saved to disk
 - ❑ Shared with others
 - ❑ Re-loaded later without redefining the model
- ❑ Get an image from test data and reshape it to match the model's expected input shape.
- ❑ Set the input tensor using `set_tensor()`.
- ❑ Call `Interpreter.invoke()` to run inference.
- ❑ Retrieve the output tensor's value.
- ❑ Convert the output into the predicted class label.



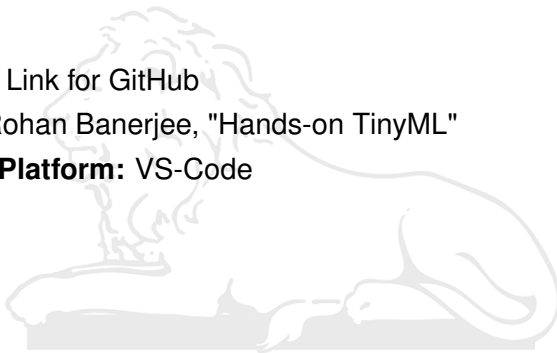
```
>>>> Baseline Model Accuracy: 91.80%
>>>> Baseline Model Prediction Time: 1.0991 seconds
>>>> Baseline model size: 2147.92 KB (2.10 MB)

>>>> TFLite Model Accuracy: 91.80%
>>>> TFLite Model Prediction Time: 0.7787 seconds
>>>> TFLite model size: 706.09 KB
```

- Here we can check that the prediction time and model size are reduced without significant change in Accuracy for MNIST dataset. Hence it is helpful in case of microcontrollers.



- ❑ **GitHub:** Link for GitHub
- ❑ **Book:** Rohan Banerjee, "Hands-on TinyML"
- ❑ **Coding Platform:** VS-Code



Thanks.